Lab Exercise 1: Simon

B Term 2025

Grading for this lab assignment can be found below in the check-offs section, in section 5. This lab is graded out of a total of 130 points, with the full breakdown shown in a table within that section.

1 Introduction

In 1978, Simon, one of the most successful electronic games, was created. The game, invented and patented by Milton Bradley and later owned by Hasbro, is an electronic spin on the classic verbal game, "Simon Says". The objective of this game tests players' visual and auditory memory, featuring four light-up buttons, each associated with a unique musical note. Once the game is started, the machine randomly plays a sequence of lights within each button and plays its corresponding note over an internal speaker. The player is tasked with correctly remembering the order of lights and repeating the sequence back to the machine by pressing the correct buttons. The longer the game continues, the faster the sequences are played, creating naturally progressing difficulty. Since its release, newer versions that innovate on the existing machine and its objectives have been released, each with its own unique gameplay that builds upon the internal electronic systems. What sets this game apart from others is its simplicity, both in playing objectives and its technical, embedded-system design. In this lab, you will implement a simplified version of the Simon game using your STM32 NUCLEO-H533RE lab board.



1.1 Learning Objectives

By the end of this lab assignment, you should be able to:

1. Design and implement circuitry for interfacing with GPIO using lab components
2. Design data structures facilitating game logic and validation
3. Interpret register and peripheral information from datasheets and user manuals
4. Diagram and implement a finite state machine
5. Design GUI components in resource-constrained environments

Please consult the Lab 0 handout if you need help connecting the Adafruit Color TFT 1.8" Shield to the lab board. The shield should connect easily and can only be attached in one orientation. Do not try to force it into the pins, as you risk damaging the shield or your lab board! If you encounter an issue with the shield that you cannot fix on your own, please ask a TA or the professor for assistance.

2 Materials

For this lab, you will need:

- STM32 NUCLEO-H533RE Lab Board
- Adafruit Color TFT 1.8" Shield
- Breadboard
- 4x push buttons
- 4x LEDs
- 4x 330 Ω resistors
- Male-to-female jumper wires
- (Optional) ECE 2010 lab kit breadboard jumper wires (for improved cable management)

3 Prelab [20]

This prelab consists of a "questions" section and a "design" section. Complete the prelab before your lab period starts and before starting work on the system specifications and requirements!

3.1 Questions [10]

These questions should be answered using the lecture notes, the STM32H533RE datasheet, and the user manual. If you use external sources to reach an answer, **be sure to cite them in your answer!**

1. Four primary configuration registers, MODER, OTYPER, OSPEEDR, and PUPDR, set the behavior of GPIO pins on our STM32. For each register,
   (a) Give a brief summary *(about 1 sentence)* of the functionality
   (b) State whether or not we are concerned with configuring that register in this class.
2. Which pin is being configured in the code below? What are its settings?

```
RCC->AHB2ENR |= RCC_AHB2ENR_GPIODEN;  // enable GPIOD clock

GPIOD->MODER &= ~GPIO_MODER_MODE12;   // ...
GPIOD->PUPDR &= ~GPIO_PUPDR_PUPD12;   // ...
GPIOD->PUPDR |=  GPIO_PUPDR_PUDP12_1; // ...
```

3. If you are going to configure GPIO pin PC1 to be an output:
   (a) what are the names of the ***specific*** GPIO control register(s) (i.e., for the particular port) that would need to be changed?
   (b) Which bit(s) in those register(s) would need to be set?
   (c) Write C code that configures the PC1 using the macros from the STM32 headers (no confusing magic numbers!).

3.2 Design [10]

READ THE ENTIRE LAB ASSIGNMENT BEFORE ATTEMPTING THESE PRELAB QUESTIONS!

1. Create a draft of a circuit diagram for the four LEDs and buttons that you will need for this lab.
   (a) LEDs **MUST** be wired in series with resistors. The LEDs have very low resistance, and the STM32's GPIO pins cannot handle the current that would be drawn by the LEDs alone. **If you connect an LED to the STM32 without a resistor, you may damage the STM32 and/or the display!**
   (b) Consult the Nucleo manual to find usable GPIO pins. Make sure you look at the table of **Morpho connector** pins; the Arduino pins are used by the display.
2. Create a draft of a state diagram for the states that model the behavior of this lab assignment. Be careful not to According to the system specifications below, you will need to determine:
   a. a list of states
   b. the **behavior** of each state — keep them as simple as possible!
   c. the actions to move between states
   d. the actions to remain within a state, if necessary

4 System Specifications and Requirements

4.1 Assignment Overview

Using the provided starter code, implement a simplified version of the Simon game on your NUCLEO-H533RE board using the included external buttons, LEDs, buzzer, and display shield in your lab kit. The starter code includes the library for interfacing with the display shield and has already configured the proper pinouts. Additionally, an interfacing library for the buzzer has been set up for you, enabling PWM signals to play a buzzer at a specified frequency. Using the information below, you will need to implement a system to determine the frequency of the notes to be played. This requirement is also documented in the starter code's comments.

Your challenge in this lab is to implement the Simon game using GPIO inputs and outputs configured manually via the control registers, not via the STM32CubeMX program. STM provides a very easy-to-use abstraction of pinout setup via the STM32CubeMX program, which will be used later in this course to set up GPIO pins; however, this lab requires you to manually set up these configurations, such that you will be able to interpret information presented in the STM32H533RE microcontroller reference manual to implement changes to our embedded system directly within code, an essential skill in the industry.

The Simon game should be implemented using four LEDs and four external buttons connected to the Nucleo via the breadboard and GPIO pins. The LEDs should light up in a random sequence, with a specific note for each LED. The same note should be used every time the same LED is lit; therefore, you will need to choose four unique notes for this project. After the sequence has been played, the user should press the corresponding buttons. If the user enters the sequence correctly, the display should show a message like "Simon says, 'That's correct!'" before playing the next sequence, adding one more LED flash each time. The sequences should start at one LED and increment to a preset limit until the user makes a mistake. If the user fails to complete a sequence correctly, the lights should quickly flash randomly while playing their associated buzzer notes, and the display should show a game-over screen.

You should implement your game as a state machine with separate states for the introduction screen, gameplay, user input handling, and game over states, among others. As with all labs in ECE 2049, the implementation of the game is up to you! As such, your state machines might include extra functionality compared to others, and your game designs might vary. As long as you meet the requirements, you should be good to go!

4.2 System Requirements [60]

To receive full credit, your Simon game should implement all of the following requirements. Questions to answer in your write-up are highlighted in yellow within their corresponding requirements:

1. The game should show a welcome screen with the word "Simon" somewhere on the display whenever the game is not in play. [5]
    a. When the game is lost, the system should return to this screen

   b. Pressing the "Select" button (pushing the joystick button down in the center on the display driver board) should return the system to this screen while in play, regardless of game state (Hint: refer to the function `TFTShield18_readButtons()` in the starter code's `main.c` and in the display driver documentation, and to the macro definitions in `Core/Inc/Graphics/Adafruit_TFTShield18_Macros.h`)
2. Pressing the "Select" button on the joystick should start the game from the welcome screen. The system should show a countdown from 3, accurate to the second, before starting the game. [5]
   a. Hint: A custom timer system is set up in your starter code that provides more accurate delays. The `DWT_delayMs(uint32_t ms)` and `DWT_delayUs(uint32_t us)` functions can be used to delay your program by a certain number of milliseconds or microseconds, respectively.
3. Four LEDs and four push buttons need to be connected to GPIO pins on the NUCLEO. [10]
   a. You will need to set up these LEDs and buttons yourself on the breadboard, connecting them to any free GPIO pins on the board.
   b. **DO NOT FORGET THE 330 Ω CURRENT LIMITING RESISTOR WITH YOUR LED!** LEDs do not resist current on their own, and the GPIO pins on your board do not have dedicated current-limiting resistors, meaning that excessive current above ∼20 mA can damage or destroy your board or the display. Given that the LED and resistor are in series with each other and GPIO on the STM32 board supplies a 3.3V potential, validate using Ohm's Law (V = IR) that the current through the resistor, and therefore the GPIO pin, does not exceed this amount using a 330 Ω resistor.[5]
   c. Use your findings from the prelab to ***MANUALLY*** configure the GPIO pins. You will not receive credit if you configure the pins using the STM32CubeMX program for this lab.
4. While in play, the game should flash a sequence on the LEDs, one LED at a time, and play a unique note on the buzzer for each LED. Each time the sequence is correctly played back on the buttons, the sequence should increment in length by one, up to at least 32 flashes. You can increase this number if you want the program to be more flexible! [15]
   - One pin of the buzzer should be connected to GPIO PE9. The other pin should be connected to ground.
   - The lab 0 starter code contains functions for controlling the buzzer: `buzzerEnable()`, `buzzerDisable()`, and `buzzerSetNote()`.
5. Create an in-game, graphical display on the Adafruit TFT display that accomplishes the following requirements: [10]
   a. Shows the button states (whether they are pressed or not) at any given point in time
      i. Each button must be represented by its own element on screen
      ii. The element should change to distinguish between button states
      iii. The element should be visually appealing and should have a similar structure to the original Simon game (ex. Simon electronic game layout).
   b. Shows the history of the last five buttons that have been pressed so far during the level
      i. The history should be represented either by small symbols or text. The color should match the color of the pressed button
   c. The current level that the player is on. This will represent their score
   d. The success message when the player gets a sequence right

6. The game should keep track of the full history of button presses to check whether the played sequence was correct. Design a data structure that can be used to track this history. Describe in your report how you implemented this history system. What is the data structure that you used to store the history, and how did you organize the history to display on screen in requirement 5?[5]
7. Upon losing the game, a "Game Over" screen should be displayed, and the lights should all flash at random very rapidly, playing their associated buzzer notes to create an annoying sound. After a certain amount of time passes, the game should return to the main menu [5]
8. The game should use the hardware Random Number Generator (RNG) peripheral to create the random sequence. This peripheral is already set up for you and can be used with the `HAL_RNG_GenerateRandomNumber(RNG_HandleTypeDef *hrng, uint32_t *random32bit)` function call.
9. A well-written, complete lab report following the write-up guidelines included below in Section 6
    a. The write-up should include a complete and updated state diagram for your game and a flow chart for your code as a whole.
    b. Be sure to include answers to the questions highlighted in yellow in Section 6 in your write-up.

4.2.1 OPTIONAL - Bonus [5]

The following requirements are bonus requirements worth a couple of extra points. They are optional and are not required to receive full credit; however, they provide extra practice with more complex systems

1. Make the game increase in speed as the level number increases. The way this is implemented is up to you and your teammate; however, it must be a noticeable in a reasonable number of rounds. [2]
2. Create a high score system that saves and displays at least the 3 highest scores. [3]

5 Check offs

Grading will be done primarily on Canvas and via internal tracking with TAs. Use the following table to track your progress within the lab assignment. Check boxes are included in case you want to manually check them off for tracking purposes, but will not be used by TAs.

Requirement	**Point Value**	**Checkbox**
Prelab	20	☐
Requirement 1: Welcome screen	5	☐
Requirement 2: Starting the game	5	☐
Requirement 3: GPIO wiring, configuration, and functionality	10	☐
Requirement 4: Gameplay	15	☐
Requirement 5: In-game display	10	☐
Requirement 6: History tracking	5	☐
Requirement 7: Game over screen	5	☐
Requirement 8: Random number generation	5	☐
Write-up	50	☐
Answers to highlighted questions in writeup	10	☐
Bonus	5	☐
Total	**140**	

6 Write Up [50]

The lab write-up must follow the guidelines in the official ECE 2049 lab report instructions document. This document is included below for ease of access and can also be found on Canvas. Remember to answer the questions highlighted in yellow in the requirements section!

The primary purpose of the report is for students to **explain and analyze** their final designs for the lab project. Exemplary reports will also **justify** important design decisions, thoughtfully **reflect** on the challenges of the project, and **evaluate** the effectiveness of the strategies used (both technical and otherwise) to navigate those challenges.

Report Style and Contents:

Your lab report should be written in a **professional** style with an attention to **technical** detail, like you might submit to a supervisor or colleague. The report should be written for an audience that is familiar with general concepts in C programming and embedded systems but is ***not* familiar with your particular hardware or the requirements of the project.** Therefore, the report should begin with thorough explanations of the project requirements, any other initial design specifications you laid out, and the functionality of your final design.

Your report should include:

- A high level description of the interface(s) and control flow of your program
- A clear, neat, and well-labeled state diagram (recommended) or flowchart
- Conceptual explanations of any peripheral settings that you configured
- Analysis and appraisal of the strengths and weaknesses of your final design and implementation: What works well, and what doesn't? Are there any improvements you might make if you had more time?
- Analysis and appraisal of the design, programming, and testing process: how did you organize the work (between members, in time, in technical components, etc.)? What challenges did you face, how did you approach them, and were your approaches effective?

Formatting and other requirements

- Neat, clear, and precise writing and formatting: you have multiple people working on the assignment – edit each others' work!
- Well-organized and labeled sections (labeled subsections are optional)
- **All figures must be numbered, captioned, and explained/referenced in the text!**
- All figures should be ***referenced and explained*** in the body text of the report. For example, it is not sufficient to include the code for a function without explaining how the function works.
- Besides a state diagram or flowchart, other figures are optional: use your best judgment; include a diagram, equation, or section of code if it makes the report easier to read and understand.
- Full line(s) of code should be treated as figures: set apart from other text, numbered, and captioned. Short "phrases" (variable names, function calls, simple mathematical expressions,

etc.) may be included inline. Ideally, use a monospaced font for code.

The rubric below will be used for grading lab reports. **You may (but are not required to) name the sections of the report according to the criteria.** Reports may earn you up to 50 points, to be added to the points listed in the sign-off checklist of individual lab assignments for your full lab grade. Points are earned by multiplying the number of points in the **Criterion** column by the percentage associated with the grader's assessment in the first row. For example, a proficient (90%) state diagram (8 points) earns 0.90*8 = 7.2 points towards the total grade. If work on the report was not distributed fairly, group members may receive adjusted final grades at the professor's discretion.

Criterion	**Exemplary (100%)**	**Proficient (90%)**	**Developing (75%)**	**Needs Improvement (60%)**
Introduction (5 points)	**Analyzes** project requirements and specifications in depth. **Contextualizes** the project within the course and within the field of embedded computing.	**Paraphrases** project requirements as given. Identifies topics relevant to the project.	Accurately outlines the scope of the project.	Describes the project incorrectly or partially.
Operation (5 points)	Clearly and thoroughly describes the interface and operation of your final design for an end user, including any bonus features.	Describes the interface and operation of your final design with few minor ambiguities or omissions.	Describes the interface and operation of your design with many ambiguities or major omissions.	Provides little or no useful guidance on how to operate the design.

Program design (10 points)	A reader of this section understands your program and **why** it is designed the way it is. **Justifies** important program design choices. **Analyzes** the operation and interoperation of each state in your state machine and any interrupt service routines, if they are present. Identifies important variables and functions in the program.	A reader of this section understands the design of your program. Thoroughly describes all states in your state machine and any interrupt service routines. Identifies important variables and functions in the program.	A reader of this section understands most of the design of your program. Describes states in your state machine and interrupt service routines in some detail. Incompletely identifies important functions and variables.	A reader of this section struggles to understand the design of your program. Lists states and interrupt service routines. Describes states and interrupt service routines incompletely or incorrectly.
State diagram or flowchart (5 points)	Neatly **illustrates the states and state transitions** in your system. All states and state transitions clearly and accurately labeled. Thoughtfully incorporates design elements like color, 2D placement, font choice, etc. to enhance readability.	Completely illustrates the states and state transitions in your system. All states and transitions are accurately labeled.	Incompletely or confusingly illustrates the states and transitions in your system. Some states and transitions are correctly labeled, but others are mislabeled, incompletely labeled, or not labeled. Some transitions may be missing.	Does not effectively illustrates states and transitions. Lacks adequate labeling or contains an incomplete set of states or a very incomplete set of transitions.
Peripherals (10 points)	For each peripheral used by the system, **analyzes** the utility of the peripheral for your design, identifies and **justifies** the choice of key parameters, and refers to the code that configures the peripheral accordingly.	For each peripheral used by the system, describes the operation of the peripheral, identifies the values of key parameters, and refers to the code that configures the peripheral accordingly.	Describes peripherals incompletely or incorrectly. Identifies some key parameters. May reference code but not explain it thoroughly.	Describes peripherals without explaining settings used in the project or describes an incomplete list of peripherals.

Discussion (10 points)	1. **Analyzes** challenges, successes, and shortcomings 2. **Describes** and evaluates strategies used to navigate challenges. Connects strategies to outcomes with evidence and sound logic. 3. Proposes improvements to the design itself **and to the design process**; convincingly **defends** proposals. 4. **Reflects** on how proposed improvements might be applied proactively to future projects.	1. **Analyzes** challenges faced and **describes** strategies used to navigate them. 2. **Analyzes** successes and shortcomings of the design with evidence. 3. Proposes improvements to the design itself to the design itself **and to the design process**.	1. **Describes** challenges, successes, and shortcomings. 2. Proposes improvements to the design.	0-1 out of 3 of the previous.
Style and formatting (5 points)	1. Includes a heading or title page with: • authors' names **and** the sections they wrote and/or edited • name of class • title of project 2. Consistent and clear writing style. 3. Well-organized, **labeled** sections. 4. Figures are easy to read, **numbered, and captioned**. 5. No more than 3 spelling or grammar mistakes. 6. Reasonable choice of fonts, margins, etc.	5 out of 6 of the previous.	3-4 out of 6 of the previous.	0-2 out of 6 of the previous.